INSTITUTO TECNOLÓGICO DE COSTA RICA

Curso: Diseño de Sistemas Digitales EL-3310

Profesor: Ing. Michael González Vega

DISEÑO E IMPLEMENTACIÓN DE UN PROCESADOR RISC-V UNICICLO

Capítulo 1 — Los cimientos

Grupo: 4

Integrantes:

Arce Quiros Iván Josué

Arias Arias Karla Priscila

Cabrera Montealegre Pablo Andrés

Fernández Murillo Chavelli

Murillo Wo Ching Julián

Fecha: 26 de octubre

1. Introducción

El presente proyecto tiene como propósito el desarrollar los componentes fundamentales de un procesador RISC-V uniciclo. En esta primera etapa se trata de implementar la unidad aritmética y lógica (ALU), un banco de registros, un generador de inmediatos y un multiplexor, los cuales constituyen elementos indispensables para integración del datapath.

2. Objetivos

2.1 Objetivo general

Diseñar, implementar y verificar funcionalmente los módulos fundamentales de un procesador RISC-V uniciclo mediante el uso de SystemVerilog, con el propósito de establecer una base para la futura integración de sus componentes en el camino de datos del procesador.

2.2 Objetivos específicos

- Implementar una unidad aritmética y lógica (ALU) capaz de realizar las operaciones de suma, resta, AND y OR con operandos de 32 bits, incorporando la señal de salida Zero para identificar cuándo el resultado es igual a cero.
- Diseñar un banco de registros compuesto por 32 registros de 32 bits, con dos puertos de lectura y un puerto de escritura, garantizando que el registro x0 conserve permanentemente el valor cero.
- Desarrollar un generador de inmediatos que permita extraer y extender correctamente los valores inmediatos de instrucciones RISC-V de tipo I y S, utilizando la señal de control ImmSrc para seleccionar el formato correspondiente.
- Implementar un multiplexor configurable que permita seleccionar entre dos señales de entrada de acuerdo con una señal de selección, facilitando el control del flujo de datos entre los componentes del procesador.
- Desarrollar bancos de prueba para cada módulo con el fin de evaluar su funcionamiento mediante casos que contemplen las operaciones, condiciones y comportamientos establecidos en las especificaciones del proyecto.

3. Organización del equipo

Integrante	Rol
Arce Quiros Ivan Josue	Verificación
Arias Arias Karla Priscila	Líder
Cabrera Montealegre Pablo Andres	Desarrollador
Fernandez Murillo Chavelli	Documentación
Murillo Wo Ching Julian	Desarrollador

4. Descripción de los componentes

4.1 Unidad aritmética y lógica (ALU)

Archivo: alu.sv

Como componente fundamental del procesador RISC-V, la Unidad Aritmético-Lógica (ALU) se encarga de todos los cálculos matemáticos y lógicos relacionados con los datos. Para este proyecto, hemos diseñado una ALU capaz de procesar dos entradas de 32 bits, denominadas A y B, junto con una señal de control (ALUControl) que especifica la operación exacta que debe ejecutarse.

La ALU admite cuatro funciones principales: suma, resta, AND a nivel de bits y OR a nivel de bits, cada una de ellas designada mediante un código de control binario específico. A la suma se le asigna 0000; a la resta, 0001;

a la operación AND, 0010; y a la operación OR, 0011. El resultado de la operación seleccionada se dirige a la salida de resultados.

Un bloque “always_comb” que emplea una sentencia “case” decodifica la señal “ALUControl” para ejecutar la operación solicitada. Si se proporciona un código de operación no definido, la salida “Result” toma por defecto el valor cero.

El módulo también cuenta con una señal de salida «Zero» que se activa en nivel alto siempre que la operación arroja un resultado igual a cero. En caso contrario, permanece en nivel bajo, lo que proporciona un mecanismo sencillo para detectar resultados nulos.

Para validar la funcionalidad de la ALU, es necesario realizar pruebas exhaustivas utilizando diversos valores de entrada, a fin de garantizar que las salidas coincidan con los resultados esperados. Asimismo, el conjunto de pruebas debe verificar el comportamiento correcto de la señal «Zero» y evaluar las operaciones que generan resultados negativos, conforme a los requisitos del proyecto.

4.2 Banco de registros (Register File)

Archivo: register_file.sv

Uno de los componentes del procesador RISC-V es el banco de registros, que almacena la información necesaria para ejecutar las instrucciones. Con el fin de permitir la lectura simultánea de dos registros y la escritura en uno, en este proyecto se implementó un banco de 32 registros, cada uno de 32 bits de ancho.

Los datos se almacenaron en un arreglo llamado “regs” que contiene los 32 registros. Se utiliza un bucle “for” para inicializar cada uno de ellos a cero al comienzo de la simulación. Esto facilita las pruebas, ya que el procedimiento comienza con valores conocidos.

La señal de reloj “clk” se utiliza para escribir datos. El módulo determina si la señal “RegWrite” está activa en un flanco positivo del reloj. Los datos de entrada “WD3” se almacenan en el registro seleccionado si dicha señal está a uno y la dirección “A3” no corresponde al registro “x0”. Si la señal “RegWrite” no está activa, no se modifica ningún registro.

La lectura combinacional se produce cuando las direcciones A1 y A2 seleccionan los registros y, sin esperar a un flanco de reloj, sus valores aparecen en RD1 y RD2. El registro x0 no admite operaciones de escritura y siempre devuelve cero.

El código utiliza “assign” para especificar las salidas de lectura, “initial” para inicializar los registros a cero, “always_ff” para controlar las escrituras mediante la señal de reloj, y “regs” para almacenar los datos.

4.3 Generador de inmediatos (Immediate Generator)

Archivo: imm_gen.sv

El generador de valores inmediatos extrae el campo de valor inmediato de una instrucción RISC-V y lo extiende hasta convertirlo en un valor de 32 bits. Para este proyecto, se implementó soporte para los formatos de tipo I y tipo S, cuya selección está controlada por la señal ImmSrc.

El diseño utiliza un bloque “always_comb” con una sentencia “case” para seleccionar el formato del valor inmediato: un valor “ImmSrc” de 00 genera un inmediato de tipo I, mientras que 01 produce un inmediato de tipo S. En ambos formatos, el bit de signo de la instrucción se replica para realizar una extensión de signo del valor a 32 bits, preservando así su signo.

La selección de un formato no admitido hace que la salida sea cero, ya que los formatos de instrucción restantes quedan fuera del alcance de esta fase inicial del proyecto.

4.4 Multiplexor (MUX)

Archivo: mux.sv

El multiplexor funciona como un selector que dirige una de dos entradas hacia la salida. En este proyecto, las entradas se denominan A y B, y ambas transportan datos de 32 bits.

La señal Select determina qué entrada se dirige a la salida Y: un valor de 0 selecciona la entrada A, mientras que un valor de 1 selecciona la entrada B. Este enrutamiento condicional se implementa mediante la instrucción “assign Y = Select ? B : A;”, lo que permite una selección puramente combinacional, sin señal de reloj.

Además, el código define un parámetro WIDTH para permitir anchos de datos configurables; sin embargo, en esta implementación, las entradas y la salida están fijadas a 32 bits. Para verificar el correcto funcionamiento del multiplexor, los casos de prueba deben evaluar ambos estados de la señal de selección (Select) a fin de garantizar que la salida Y coincida con la entrada correspondiente.

5. Estrategia de verificación

[Expliquen cómo se diseñó y ejecutó cada testbench, cómo se compararon los resultados esperados con los obtenidos y cómo se identifican PASS/FAIL.]

- ALU: ADD, SUB, AND, OR, resultado cero y al menos un resultado negativo.
- Register File: escritura/lectura, dos lecturas y comportamiento de x0.
- Immediate Generator: al menos dos casos I-type y dos S-type, incluyendo extensión de signo.
- MUX: selección de cada una de las dos entradas.

6. Resultados obtenidos

ALU:

```
[PASS] ALU-01 ADD 5+3
  ENTRADAS:
    A: DEC=5 | HEX=0x00000005
       BIN=00000000000000000000000000000101
    B: DEC=3 | HEX=0x00000003
       BIN=00000000000000000000000000000011
  RESULTADO ESPERADO:
    DEC=8 | HEX=0x00000008
    BIN=00000000000000000000000000001000 | Zero=0
  RESULTADO OBTENIDO:
    DEC=8 | HEX=0x00000008
    BIN=00000000000000000000000000001000 | Zero=0
----------------------------------------
[PASS] ALU-02 SUB 5-3
  ENTRADAS:
    A: DEC=5 | HEX=0x00000005
       BIN=00000000000000000000000000000101
    B: DEC=3 | HEX=0x00000003
       BIN=00000000000000000000000000000011
  RESULTADO ESPERADO:
    DEC=2 | HEX=0x00000002
    BIN=00000000000000000000000000000010 | Zero=0
  RESULTADO OBTENIDO:
    DEC=2 | HEX=0x00000002
    BIN=00000000000000000000000000000010 | Zero=0
----------------------------------------
[PASS] ALU-03 AND
  ENTRADAS:
    A: DEC=-252645136 | HEX=0xf0f0f0f0
       BIN=11110000111100001111000011110000
    B: DEC=267390960 | HEX=0x0ff00ff0
       BIN=00001111111100000000111111110000
  RESULTADO ESPERADO:
    DEC=15728880 | HEX=0x00f000f0
    BIN=00000000111100000000000011110000 | Zero=0
  RESULTADO OBTENIDO:
    DEC=15728880 | HEX=0x00f000f0
    BIN=00000000111100000000000011110000 | Zero=0
----------------------------------------
[PASS] ALU-04 OR
  ENTRADAS:
    A: DEC=-252645136 | HEX=0xf0f0f0f0
       BIN=11110000111100001111000011110000
    B: DEC=267390960 | HEX=0x0ff00ff0
       BIN=00001111111100000000111111110000
  RESULTADO ESPERADO:
    DEC=-983056 | HEX=0xfff0fff0
    BIN=11111111111100001111111111110000 | Zero=0
  RESULTADO OBTENIDO:
    DEC=-983056 | HEX=0xfff0fff0
    BIN=11111111111100001111111111110000 | Zero=0
----------------------------------------
[PASS] ALU-05 SUB 5-5 (Zero=1)
  ENTRADAS:
    A: DEC=5 | HEX=0x00000005
       BIN=00000000000000000000000000000101
    B: DEC=5 | HEX=0x00000005
       BIN=00000000000000000000000000000101
  RESULTADO ESPERADO:
    DEC=0 | HEX=0x00000000
    BIN=00000000000000000000000000000000 | Zero=1
  RESULTADO OBTENIDO:
    DEC=0 | HEX=0x00000000
    BIN=00000000000000000000000000000000 | Zero=1
----------------------------------------
[PASS] ALU-06 SUB 3-5 (negativo)
  ENTRADAS:
    A: DEC=3 | HEX=0x00000003
       BIN=00000000000000000000000000000011
    B: DEC=5 | HEX=0x00000005
       BIN=00000000000000000000000000000101
  RESULTADO ESPERADO:
    DEC=-2 | HEX=0xfffffffe
    BIN=11111111111111111111111111111110 | Zero=0
  RESULTADO OBTENIDO:
    DEC=-2 | HEX=0xfffffffe
    BIN=11111111111111111111111111111110 | Zero=0
----------------------------------------
[PASS] ALU-07 ADD overflow (Zero=1)
  ENTRADAS:
    A: DEC=-1 | HEX=0xffffffff
       BIN=11111111111111111111111111111111
    B: DEC=1 | HEX=0x00000001
       BIN=00000000000000000000000000000001
  RESULTADO ESPERADO:
    DEC=0 | HEX=0x00000000
    BIN=00000000000000000000000000000000 | Zero=1
  RESULTADO OBTENIDO:
    DEC=0 | HEX=0x00000000
    BIN=00000000000000000000000000000000 | Zero=1
----------------------------------------
[PASS] ALU-08 AND disjunto (Zero=1)
  ENTRADAS:
    A: DEC=-1431655766 | HEX=0xaaaaaaaa
       BIN=10101010101010101010101010101010
    B: DEC=1431655765 | HEX=0x55555555
       BIN=01010101010101010101010101010101
  RESULTADO ESPERADO:
    DEC=0 | HEX=0x00000000
    BIN=00000000000000000000000000000000 | Zero=1
  RESULTADO OBTENIDO:
    DEC=0 | HEX=0x00000000
    BIN=00000000000000000000000000000000 | Zero=1
----------------------------------------

========== RESUMEN ALU ==========
Pruebas ejecutadas: 8
Pruebas aprobadas:  8
Pruebas fallidas:   0
Estado general: PASS
================================
testbench.sv:190: $finish called at 80000 (1ps)
Done
```

Imm_gen:

```
[PASS] IMM-01 I-type positivo (+100)
  ENTRADAS:
    Instr: DEC=105054867 | HEX=0x06430293
           BIN=00000110010000110000001010010011
    ImmSrc=00
  RESULTADO ESPERADO:
    DEC=100 | HEX=0x00000064
    BIN=00000000000000000000000001100100
  RESULTADO OBTENIDO:
    DEC=100 | HEX=0x00000064
    BIN=00000000000000000000000001100100
----------------------------------------
[PASS] IMM-02 I-type negativo (-20)
  ENTRADAS:
    Instr: DEC=-20774253 | HEX=0xfec30293
           BIN=11111110110000110000001010010011
    ImmSrc=00
  RESULTADO ESPERADO:
    DEC=-20 | HEX=0xffffffec
    BIN=11111111111111111111111111101100
  RESULTADO OBTENIDO:
    DEC=-20 | HEX=0xffffffec
    BIN=11111111111111111111111111101100
----------------------------------------
[PASS] IMM-03 S-type positivo (+100)
  ENTRADAS:
    Instr: DEC=108208675 | HEX=0x06732223
           BIN=00000110011100110010001000100011
    ImmSrc=01
  RESULTADO ESPERADO:
    DEC=100 | HEX=0x00000064
    BIN=00000000000000000000000001100100
  RESULTADO OBTENIDO:
    DEC=100 | HEX=0x00000064
    BIN=00000000000000000000000001100100
----------------------------------------
[PASS] IMM-04 S-type negativo (-20)
  ENTRADAS:
    Instr: DEC=-26008029 | HEX=0xfe732623
           BIN=11111110011100110010011000100011
    ImmSrc=01
  RESULTADO ESPERADO:
    DEC=-20 | HEX=0xffffffec
    BIN=11111111111111111111111111101100
  RESULTADO OBTENIDO:
    DEC=-20 | HEX=0xffffffec
    BIN=11111111111111111111111111101100
----------------------------------------
[PASS] IMM-05 I-type maximo (+2047)
  ENTRADAS:
    Instr: DEC=2146632339 | HEX=0x7ff30293
           BIN=01111111111100110000001010010011
    ImmSrc=00
  RESULTADO ESPERADO:
    DEC=2047 | HEX=0x000007ff
    BIN=00000000000000000000011111111111
  RESULTADO OBTENIDO:
    DEC=2047 | HEX=0x000007ff
    BIN=00000000000000000000011111111111
----------------------------------------
[PASS] IMM-06 I-type minimo (-2048)
  ENTRADAS:
    Instr: DEC=-2147286381 | HEX=0x80030293
           BIN=10000000000000110000001010010011
    ImmSrc=00
  RESULTADO ESPERADO:
    DEC=-2048 | HEX=0xfffff800
    BIN=11111111111111111111100000000000
  RESULTADO OBTENIDO:
    DEC=-2048 | HEX=0xfffff800
    BIN=11111111111111111111100000000000
----------------------------------------
[PASS] IMM-07 S-type maximo (+2047)
  ENTRADAS:
    Instr: DEC=2121478051 | HEX=0x7e732fa3
           BIN=01111110011100110010111110100011
    ImmSrc=01
  RESULTADO ESPERADO:
    DEC=2047 | HEX=0x000007ff
    BIN=00000000000000000000011111111111
  RESULTADO OBTENIDO:
    DEC=2047 | HEX=0x000007ff
    BIN=00000000000000000000011111111111
----------------------------------------
[PASS] IMM-08 S-type minimo (-2048)
  ENTRADAS:
    Instr: DEC=-2139938781 | HEX=0x80732023
           BIN=10000000011100110010000000100011
    ImmSrc=01
  RESULTADO ESPERADO:
    DEC=-2048 | HEX=0xfffff800
    BIN=11111111111111111111100000000000
  RESULTADO OBTENIDO:
    DEC=-2048 | HEX=0xfffff800
    BIN=11111111111111111111100000000000
----------------------------------------

========== RESUMEN IMM GEN ==========
Pruebas ejecutadas: 8
Pruebas aprobadas:  8
Pruebas fallidas:   0
Estado general: PASS
==================================
testbench.sv:205: $finish called at 80000 (1ps)
Done
```

MUX:

```
[PASS] MUX-01 Select=0 (Entrada A)
  ENTRADAS:
    A: DEC=305419896 | HEX=0x12345678
       BIN=00010010001101000101011001111000
    B: DEC=-1412567296 | HEX=0xabcdef00
       BIN=10101011110011011110111100000000
    Select=0
  RESULTADO ESPERADO:
    DEC=305419896 | HEX=0x12345678
    BIN=00010010001101000101011001111000
  RESULTADO OBTENIDO:
    DEC=305419896 | HEX=0x12345678
    BIN=00010010001101000101011001111000
----------------------------------------
[PASS] MUX-02 Select=1 (Entrada B)
  ENTRADAS:
    A: DEC=305419896 | HEX=0x12345678
       BIN=00010010001101000101011001111000
    B: DEC=-1412567296 | HEX=0xabcdef00
       BIN=10101011110011011110111100000000
    Select=1
  RESULTADO ESPERADO:
    DEC=-1412567296 | HEX=0xabcdef00
    BIN=10101011110011011110111100000000
  RESULTADO OBTENIDO:
    DEC=-1412567296 | HEX=0xabcdef00
    BIN=10101011110011011110111100000000
----------------------------------------
[PASS] MUX-03 Select=0 (Entrada A)
  ENTRADAS:
    A: DEC=286331153 | HEX=0x11111111
       BIN=00010001000100010001000100010001
    B: DEC=-1431655766 | HEX=0xaaaaaaaa
       BIN=10101010101010101010101010101010
    Select=0
  RESULTADO ESPERADO:
    DEC=286331153 | HEX=0x11111111
    BIN=00010001000100010001000100010001
  RESULTADO OBTENIDO:
    DEC=286331153 | HEX=0x11111111
    BIN=00010001000100010001000100010001
----------------------------------------
[PASS] MUX-04 Select=1 (Entrada B)
  ENTRADAS:
    A: DEC=286331153 | HEX=0x11111111
       BIN=00010001000100010001000100010001
    B: DEC=-1431655766 | HEX=0xaaaaaaaa
       BIN=10101010101010101010101010101010
    Select=1
  RESULTADO ESPERADO:
    DEC=-1431655766 | HEX=0xaaaaaaaa
    BIN=10101010101010101010101010101010
  RESULTADO OBTENIDO:
    DEC=-1431655766 | HEX=0xaaaaaaaa
    BIN=10101010101010101010101010101010
----------------------------------------
[PASS] MUX-05 Select=0 (Todos ceros)
  ENTRADAS:
    A: DEC=0 | HEX=0x00000000
       BIN=00000000000000000000000000000000
    B: DEC=-1 | HEX=0xffffffff
       BIN=11111111111111111111111111111111
    Select=0
  RESULTADO ESPERADO:
    DEC=0 | HEX=0x00000000
    BIN=00000000000000000000000000000000
  RESULTADO OBTENIDO:
    DEC=0 | HEX=0x00000000
    BIN=00000000000000000000000000000000
----------------------------------------
[PASS] MUX-06 Select=1 (Todos unos)
  ENTRADAS:
    A: DEC=0 | HEX=0x00000000
       BIN=00000000000000000000000000000000
    B: DEC=-1 | HEX=0xffffffff
       BIN=11111111111111111111111111111111
    Select=1
  RESULTADO ESPERADO:
    DEC=-1 | HEX=0xffffffff
    BIN=11111111111111111111111111111111
  RESULTADO OBTENIDO:
    DEC=-1 | HEX=0xffffffff
    BIN=11111111111111111111111111111111
----------------------------------------

========== RESUMEN MUX ==========
Pruebas ejecutadas: 6
Pruebas aprobadas:  6
Pruebas fallidas:   0
Estado general: PASS
==================================
testbench.sv:173: $finish called at 60000 (1ps)
Done
```

7. Matriz de Verificación

ID	Módulo	Prueba	Resultado esperado	Resultado obtenido	Estado
ALU-01	ALU	Sumar 5 + 3 (ADD 5+3)	8 (0x00000008)	8 (0x00000008, Zero=0)	PASS
ALU-02	ALU	Restar 5 - 3 (SUB 5-3)	2 (0x00000002)	2 (0x00000002, Zero=0)	PASS
ALU-03	ALU	Operación AND (AND)	15728880 (0x00f00f0)	15728880 (0x00f00f0, Zero=0)	PASS
ALU-04	ALU	Operación OR (OR)	-983056 (0xfff0fff0)	-983056 (0xfff0fff0, Zero=0)	PASS
ALU-05	ALU	Restar 5 - 5 (SUB 5-5)	0 y Zero = 1	0 y Zero = 1	PASS
ALU-06	ALU	Restar 3 - 5 (SUB 3-5)	-2 (0xfffffffe)	-2 (0xfffffffe, Zero=0)	PASS
ALU-07	ALU	Suma con desbordamiento (ADD overflow)	0 y Zero = 1	0 y Zero = 1	PASS
ALU-08	ALU	AND con operandos disjuntos	0 y Zero = 1	0 y Zero = 1	PASS
RF-01	Register File	Escribir y leer el registro 1	Leer el valor escrito	15 (valor de referencia previo)	PASS
IMM-01	Immediate Generator	I-type positivo (+100)	100 (0x00000064)	100 (0x00000064)	PASS
IMM-02	Immediate Generator	I-type negativo (-20)	-20 (0xffffffec)	-20 (0xffffffec)	PASS
IMM-03	Immediate Generator	S-type positivo (+100)	100 (0x00000064)	100 (0x00000064)	PASS
IMM-04	Immediate Generator	S-type negativo (-20)	-20 (0xffffffec)	-20 (0xffffffec)	PASS
IMM-05	Immediate Generator	I-type máximo (+2047)	2047 (0x000007ff)	2047 (0x000007ff)	PASS
IMM-06	Immediate Generator	I-type mínimo (-2048)	-2048 (0xfffff800)	-2048 (0xfffff800)	PASS

IMM-07	Immediate Generator	S-type máximo (+2047)	2047 (0x000007ff)	2047 (0x000007ff)	PASS
IMM-08	Immediate Generator	S-type mínimo (-2048)	-2048 (0xfffff800)	-2048 (0xfffff800)	PASS
MUX-01	MUX	Seleccionar entrada A (Select=0)	305419896 (0x12345678)	305419896 (0x12345678)	PASS
MUX-02	MUX	Seleccionar entrada B (Select=1)	-1412567296 (0xabcdef00)	-1412567296 (0xabcdef00)	PASS
MUX-03	MUX	Seleccionar entrada A (Select=0)	286331153 (0x1111111)	286331153 (0x1111111)	PASS
MUX-04	MUX	Seleccionar entrada B (Select=1)	-1431655766 (0xaaaaaaaal)	-1431655766 (0xaaaaaaaal)	PASS
MUX-05	MUX	Select=0 (todos ceros)	0 (0x00000000)	0 (0x00000000)	PASS
MUX-06	MUX	Select=1 (todos unos)	-1 (0xffffffff)	-1 (0xffffffff)	PASS

8. Conclusiones

A lo largo del desarrollo de este proyecto, se implementaron con éxito, mediante SystemVerilog, los módulos principales para la etapa inicial del procesador RISC-V. Este proceso permitió una comprensión más profunda de la funcionalidad de cada componente y de las relaciones entre los distintos módulos. Asimismo, puso de relieve la importancia fundamental de realizar pruebas rigurosas para detectar errores y garantizar que el diseño cumpla con las especificaciones del proyecto. En conjunto, este trabajo sienta una base sólida para futuras etapas de desarrollo e integración del procesador.

9. Referencias

Harris, D. M., & Harris, S. L. (2021). Digital design and computer architecture: RISC-V edition. Morgan Kaufmann.

Mano, M. M., & Ciletti, M. D. (2017). Digital design (6th ed.). Pearson.

Palnitkar, S. (2003). Verilog HDL: A guide to digital design and synthesis (2nd ed.). Prentice Hall.

Patterson, D. A., & Hennessy, J. L. (2021). Computer organization and design RISC-V edition: The hardware/software interface (2nd ed.). Morgan Kaufmann.